

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Sistemas Distribuidos				
TÍTULO DE LA PRÁCTICA:	Seguridad Informática en Sistemas Distribuidos				
NÚMERO DE LA PRÁCTICA:	11	AÑO LECTIVO:	2026	NRO. SEMESTRE:	A
FECHA DE PRESENTACIÓN:	30/06/2026	HORA DE PRESENTACIÓN:	11:59:00		
INTEGRANTE(s): - Bedregal Perez Daniel - Jara Mamani Mariel Alisson - Mestas Zegarra Christian Raul - Noa Camino Yenaro Joel - Sequeiros Condori Luis Gustavo				NOTA (0 - 20): 	Nota colocada por el docente
DOCENTE: Mg. Maribel Molina Barriga					

RESULTADOS Y PRUEBAS				
Actividad 1: Identificación de Amenazas Para el caso empresarial de LogiMarket Perú S.A.C., se realiza un análisis de amenazas siguiendo la metodología del NIST Cybersecurity Framework [1] y las categorías de riesgo del OWASP API Security Top 10 [2]. Se identifican los activos críticos de la plataforma, las amenazas potenciales asociadas, las vulnerabilidades subyacentes, el impacto potencial y el nivel de riesgo resultante.				
Activo	Amenaza	Vulnerabilidad	Impacto	Riesgo
Servicio de Autenticación	Broken Authentication (API2), credential stuffing, fuerza bruta [2]	Ausencia de MFA, contraseñas débiles, tokens JWT sin validación de expiración	Compromiso total de cuentas de usuarios, acceso no autorizado a datos personales y financieros	Crítico
Servicio de Inventario	Inyección SQL, broken object-level authorization (BOLA) [2]	Falta de validación de parámetros en endpoints REST, ausencia de RBAC por objeto	Manipulación de stock, pérdida de integridad de datos de inventario, sobreventa	Alto
Servicio de Pagos	Exposición de datos sensibles, interceptación de tráfico entre servicios	Comunicación sin cifrado entre microservicios, almacenamiento de datos de pago en texto plano	Pérdida de información financiera, fraude en transacciones, multas regulatorias	Crítico
Servicio de Logística	Broken access control, acceso no autorizado a información de envíos	Ausencia de control de acceso basado en roles, credenciales compartidas entre empleados	Acceso no autorizado a rutas de envío, manipulación de estados de entrega	Alto

Portal Web para Clientes	XSS, CSRF, server-side request forgery (SSRF) [2]	Falta de validación de entrada en formularios, ausencia de headers de seguridad	Robo de sesiones de clientes, inyección de código malicioso, robo de credenciales	Alto
Aplicación Móvil	Improper inventory management (API9), exposición de endpoints deprecated	API keys embebidas en el código fuente, versiones obsoletas sin parches de seguridad	Acceso a funcionalidades no autorizadas, extracción de datos del cliente	Alto

Figura 1: Matriz de riesgos de seguridad para los activos críticos de LogiMarket Perú S.A.C.

El análisis revela que la combinación de autenticación débil, comunicación sin cifrar entre servicios y ausencia de mecanismos de auditoría constituye una superficie de ataque significativa. Según el OWASP API Security Top 10 [2], los errores de autenticación y autorización representan las categorías de riesgo más prevalentes en APIs empresariales. La propuesta de seguridad integral para LogiMarket aborda estos vectores mediante autenticación multifactor, cifrado TLS y monitoreo continuo, como se detalla en las siguientes actividades.

Actividad 2: Diseño de Autenticación Segura

Para LogiMarket Perú S.A.C., se diseña una arquitectura de autenticación que aborde los incidentes de accesos no autorizados y credenciales compartidas, siguiendo las directrices del NIST Cybersecurity Framework [1] y el patrón de autenticación a nivel de borde (edge-level) recomendado por [3].

Arquitectura Propuesta

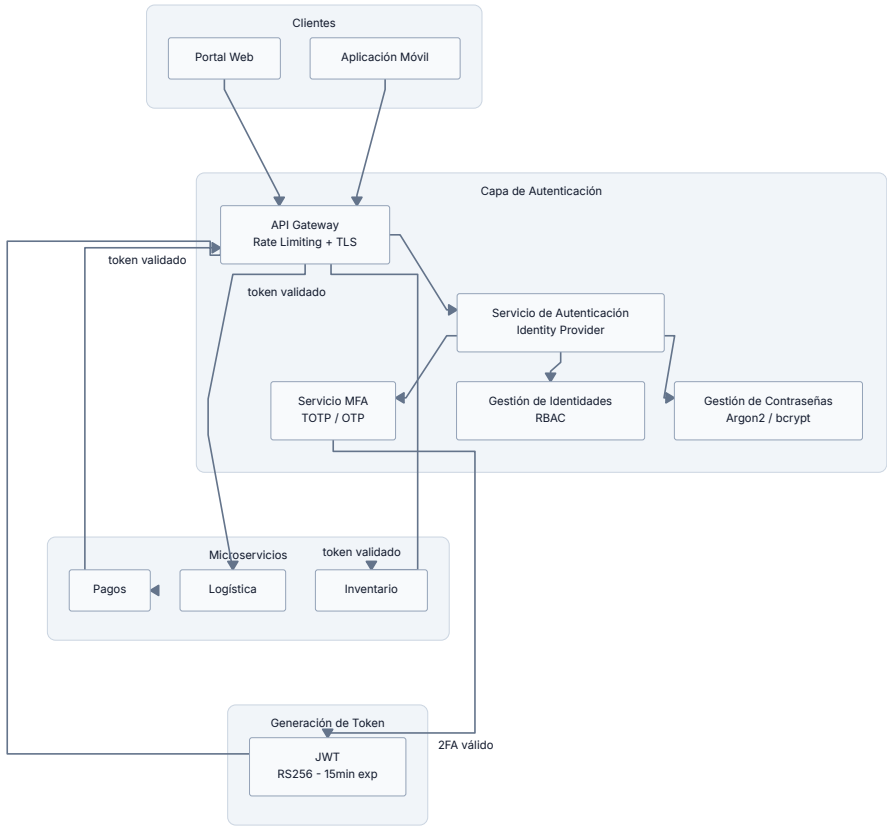


Figura 1: Diagrama de arquitectura de autenticación para LogiMarket Perú S.A.C.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

Componentes Principales

1. Portal Web y Aplicación Móvil

Son los puntos de acceso de los usuarios a la plataforma. Desde estos clientes se envían las solicitudes de autenticación y posteriormente se consumen los servicios de negocio disponibles en la arquitectura de microservicios. Cada cliente implementa flujos de autenticación seguros que incluyen validación de certificados TLS y protección contra replay attacks.

2. API Gateway

Actúa como punto de entrada único para todas las solicitudes provenientes de los clientes, siguiendo el patrón de API Gateway documentado por [3]. Su función principal es canalizar las peticiones hacia el Servicio de Autenticación y posteriormente hacia los microservicios correspondientes, permitiendo centralizar aspectos de seguridad y control.

3. Servicio de Autenticación (Identity Provider)

Es el componente encargado de validar las credenciales de los usuarios y gestionar el proceso completo de autenticación. Coordina la interacción con los servicios de gestión de identidades, contraseñas y autenticación multifactor. Utiliza el flujo OAuth 2.0 con PKCE para garantizar un alto nivel de seguridad [2].

4. Gestión de Identidades (RBAC)

Administra la información de usuarios, roles y permisos dentro de la organización. Permite aplicar controles de acceso basados en roles (RBAC), garantizando que cada usuario tenga acceso únicamente a los recursos que le corresponden según sus funciones. Los roles definidos incluyen: admin, user y guest, con permisos jerárquicos.

5. Gestión de Contraseñas

Se encarga del almacenamiento seguro de las credenciales utilizando algoritmos de hash como Argon2 o bcrypt. Implementa políticas de complejidad (mínimo 12 caracteres, combinación de mayúsculas, números y símbolos), recuperación de contraseñas por email y mecanismos para prevenir el uso de credenciales débiles o comprometidas [4].

6. Servicio MFA

Implementa la autenticación multifactor mediante códigos TOTP (Time-Based One-Time Password) compatible con aplicaciones como Google Authenticator y Authy. El segundo factor añade una capa adicional de seguridad que reduce significativamente el riesgo de accesos no autorizados, incluso cuando las credenciales primarias son comprometidas [5].

7. JWT (JSON Web Token)

Una vez completada la autenticación, el sistema genera un token JWT firmado digitalmente con el algoritmo RS256 que contiene información sobre la identidad y permisos del usuario. El token incluye claims estándar (sub, iss, exp, iat, roles) y tiene una expiración de 15 minutos para tokens de acceso y 7 días para refresh tokens.

Flujo de Autenticación

1. El usuario ingresa credenciales en el Portal Web o Aplicación Móvil.
2. La solicitud es enviada al Servicio de Autenticación a través del API Gateway.
3. El Servicio de Autenticación valida las credenciales mediante la Gestión de Contraseñas.
4. Se verifica la identidad y los permisos del usuario mediante la Gestión de Identidades.
5. Se solicita y valida el segundo factor de autenticación mediante el Servicio MFA.
6. Una vez completada la validación, se genera un token JWT con expiración controlada.
7. El usuario utiliza el token para acceder a los microservicios de Inventario, Pagos y Logística.

La combinación de estos mecanismos mitiga los vectores de ataque identificados en la matriz de riesgos, incluyendo broken authentication (API2), credential stuffing y ataques de fuerza bruta, como se documenta en el OWASP API Security Top 10 [2].

Actividad 3: Seguridad de Comunicaciones

Se implementa una demostración completa de seguridad de comunicaciones para LogiMarket Perú S.A.C., incluyendo generación de certificados TLS, configuración de canal HTTPS, despliegue de API con métricas de seguridad y stack de monitoreo y auditoría. El sistema utiliza Docker Compose para orquestar todos los componentes.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

Generación de Certificados TLS Autofirmados

Se genera un certificado autofirmado RSA de 4096 bits utilizando OpenSSL con Subject Alternative Names (SAN) para múltiples dominios locales. El script `generate-certs.sh` automatiza el proceso de generación:

```
echo "► Generando clave privada RSA 4096 bits..."
openssl genrsa -out "$KEY_FILE" 4096 2>/dev/null

chmod 600 "$KEY_FILE"

echo "► Generando certificado autofirmado (CN=$CN, SAN=$SAN, $DAYS días)..."

SAN_ENV="subjectAltName=${SAN}"
openssl req -new -x509 \
  -key "$KEY_FILE" \
  -out "$CERT_FILE" \
  -days "$DAYS" \
  -subj "/C=PE/ST=Arequipa/L=Arequipa/O=SD-Lab/OU=Lab11/CN=$CN" \
  -addext "$SAN_ENV" \
  -addext "keyUsage=digitalSignature,keyEncipherment" \
  -addext "extendedKeyUsage=serverAuth,clientAuth" 2>/dev/null
```

El certificado generado incluye los campos `keyUsage=digitalSignature,keyEncipherment` y `extendedKeyUsage=serverAuth,clientAuth`, permitiendo su uso tanto para servidor como para cliente. Los SAN incluyen `DNS:localhost`, `DNS:traefik.localhost`, `IP:127.0.0.1` e `IP:::1` para compatibilidad completa con los diferentes endpoints de acceso.

API Go con Métricas de Seguridad

La aplicación principal es un servidor HTTP en Go que implementa un middleware de métricas para registrar cada request con su nivel de severidad (info, warn, error), método HTTP, path, status code y latencia. Este enfoque permite correlacionar métricas de rendimiento con eventos de seguridad:

```
func metricsMiddleware(next http.HandlerFunc) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        mr := &metricsResponse{ResponseWriter: w, status: http.StatusOK}
        next(mr, r)
        duration := time.Since(start).Seconds()
        level := "info"
        if mr.status >= 500 {
            level = "error"
        } else if mr.status >= 400 {
            level = "warn"
        }
        log.Printf(`{"level":"%s","msg":"%s %s %d %s","method":"%s","path":"%s","status":%d,"duration":%.3f}`, level, r.Method,
            r.URL.Path, mr.status, duration, r.Method, r.URL.Path, mr.status, duration)
        recordMetrics(r.Method, r.URL.Path, mr.status, duration)
    }
}
```

El endpoint `/metrics` expone métricas en formato Prometheus incluyendo `http_requests_total`, `http_request_errors_total`, `http_request_duration_seconds` (histograma) y `process_uptime_seconds`, permitiendo el monitoreo continuo de la salud del servicio.

Los endpoints disponibles incluyen rutas protegidas para productos y usuarios, así como un endpoint de prueba de errores 500:

```
func main() {
    log.SetFlags(0)
    mux := http.NewServeMux()
    mux.HandleFunc("GET /", metricsMiddleware(index))
    mux.HandleFunc("GET /health", metricsMiddleware(health))
    mux.HandleFunc("GET /api/info", metricsMiddleware(info))
    mux.HandleFunc("GET /api/products", metricsMiddleware(listProducts))
    mux.HandleFunc("GET /api/products/{id}", metricsMiddleware(getProduct))
    mux.HandleFunc("GET /api/users", metricsMiddleware(listUsers))
    mux.HandleFunc("POST /api/echo", metricsMiddleware(echo))
    mux.HandleFunc("GET /api/500", metricsMiddleware(errorTest))
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

```

mux.HandleFunc("GET /metrics", metricsHandler)

addr := ":8080"
log.Printf("SD-Lab11 app listening on %s", addr)
if err := http.ListenAndServe(addr, mux); err != nil {
    log.Fatal(err)
}
}

```

Configuración de Traefik Reverse Proxy

Traefik v3.7 funciona como API Gateway y reverse proxy, implementando redirección automática HTTP a HTTPS, terminación TLS y descubrimiento dinámico de servicios a través del Docker socket. La configuración incluye entypoints para HTTP (puerto 80), HTTPS (puerto 443) y el dashboard de Traefik (puerto 8080):

```

entryPoints:
  web:
    address: ":80"
  http:
    redirections:
      entryPoint:
        to: websecure
        scheme: https
        permanent: true
  websecure:
    address: ":443"
  traefik:
    address: ":8080"

providers:
  docker:
    endpoint: "unix:///var/run/docker.sock"
    exposedByDefault: false
    network: web
  file:
    filename: /etc/traefik/dynamic/tls.yml
    watch: true

log:
  level: INFO
  format: common

```

La redirección HTTP→HTTPS se configura a nivel de endpoint con `permanent: true`, garantizando que todas las solicitudes no cifradas sean redirigidas automáticamente al canal seguro. El certificado TLS se carga dinámicamente desde el volumen `/certs/` montado como `read-only`.

Orquestación con Docker Compose

El stack completo se despliega mediante Docker Compose con dos servicios principales: Traefik y la aplicación Go, conectados a través de una red Docker dedicada:

```

services:
  traefik:
    image: traefik:v3.7
    container_name: sd-lab11-traefik
    restart: unless-stopped
    networks:
      - web
    ports:
      - "80:80"
      - "443:443"
      - "8080:8080"
    command:
      - "--configFile=/etc/traefik/traefik.yml"
      - "--entryPoints.traefik.address=:8080"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - ./traefik/traefik.yml:/etc/traefik/traefik.yml:ro

```

```

- ./traefik/dynamic:/etc/traefik/dynamic:ro
- ./certs:/certs:ro
labels:
- "traefik.enable=true"
- "traefik.docker.network=web"
- "traefik.http.routers.dashboard.rule=Host(`traefik.localhost`)"
- "traefik.http.routers.dashboard.entrypoints=websecure"
- "traefik.http.routers.dashboard.tls=true"
- "traefik.http.routers.dashboard.service=api@internal"
- "traefik.http.services.dashboard.loadbalancer.server.port=8080"

app:
build:
context: ./app
container_name: sd-lab11-app
restart: unless-stopped
networks:
- web
labels:
- "traefik.enable=true"
- "traefik.docker.network=web"
- "traefik.http.routers.app.rule=Host(`localhost`)"
- "traefik.http.routers.app.entrypoints=websecure"
- "traefik.http.routers.app.tls=true"
- "traefik.http.routers.app.priority=10"
- "traefik.http.services.app.loadbalancer.server.port=8000"

networks:
web:
name: sd-lab11-web

```

Los labels de Traefik en cada contenedor definen las reglas de enrutamiento: Host(`localhost`) para la aplicación y Host(`traefik.localhost`) para el dashboard, ambos en el endpoint websecure (HTTPS) con TLS habilitado.

Stack de Monitoreo y Auditoría

El sistema implementa un stack completo de observabilidad con Prometheus para métricas, Grafana Alloy para recolección de logs, Loki para almacenamiento y Grafana para visualización. La configuración detallada de cada componente se presenta en la Actividad 5.

Evidencias de Implementación

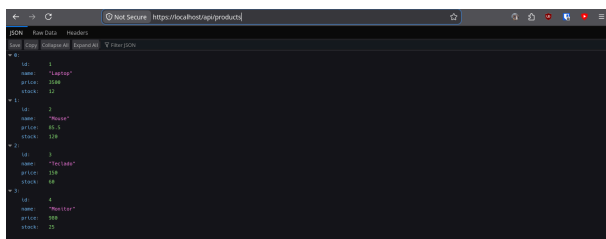


Figura 2: Acceso HTTPS a la API en <https://localhost/api/products>

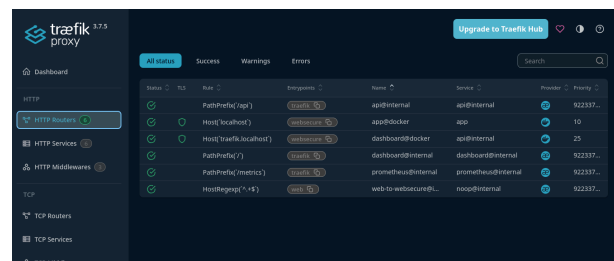


Figura 3: Dashboard de Traefik mostrando routers HTTPS configurados

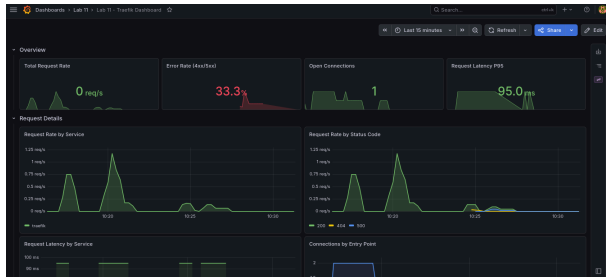


Figura 4: Dashboard de Traefik en Grafana con métricas de requests por servicio y status code

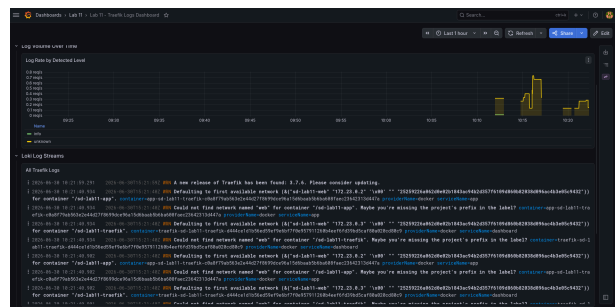


Figura 5: Panel de logs de Traefik en Loki con correlación temporal

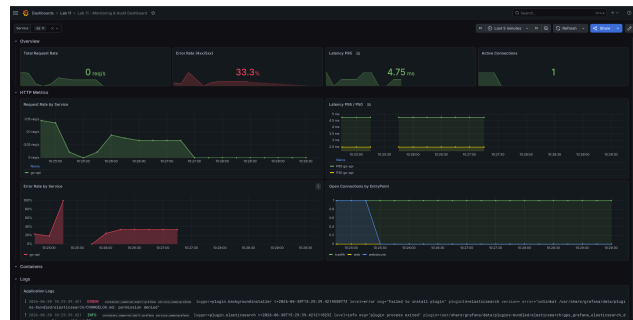


Figura 6: Dashboard unificado de Grafana con métricas de requests, latencia, errores y logs

Las capturas demuestran que el sistema opera correctamente: la aplicación Go responde vía HTTPS con datos JSON, Traefik gestiona el enrutamiento y la terminación TLS, y Grafana muestra un dashboard unificado que correlaciona métricas de Prometheus con logs de Loki, proporcionando visibilidad completa sobre el estado de seguridad del sistema.

Actividad 4: Protección de APIs

Para LogiMarket Perú S.A.C., se diseña una arquitectura segura para APIs empresariales que integra mecanismos de autenticación, autorización, control de tráfico y monitoreo, siguiendo las mejores prácticas documentadas por el OWASP API Security Top 10 [2] y las directrices de NIST para arquitecturas de microservicios [1].

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

Arquitectura Propuesta

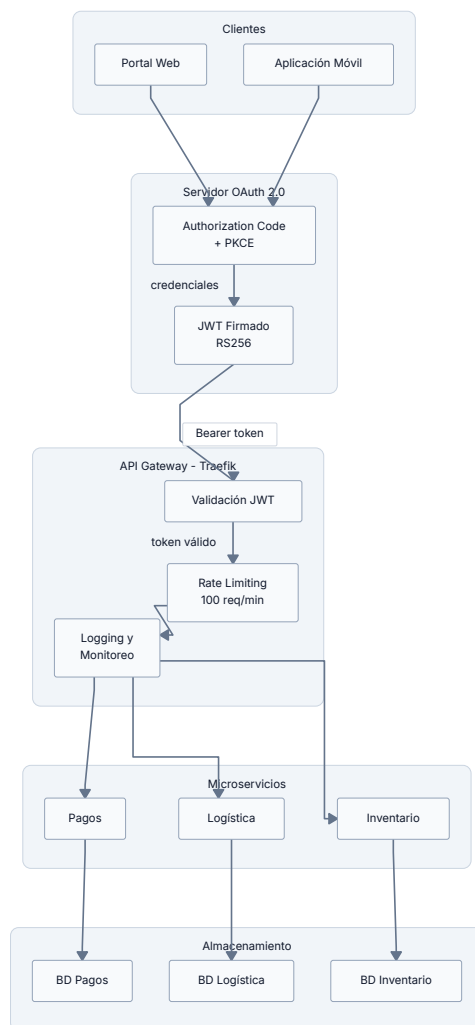


Figura 7: Diagrama de arquitectura segura para APIs empresariales de LogiMarket

Componentes Principales

1. Portal Web y Aplicación Móvil

Son los clientes que consumen los servicios empresariales. Desde estos sistemas los usuarios realizan operaciones relacionadas con compras, pagos, inventario y seguimiento logístico. Cada cliente implementa flujos de autenticación OAuth 2.0 con PKCE para garantizar la seguridad en la obtención de tokens.

2. Servidor OAuth 2.0

Es el componente responsable de autenticar a los usuarios y otorgar autorización para acceder a los recursos protegidos. Se implementa el flujo Authorization Code con PKCE (Proof Key for Code Exchange) para proporcionar un alto nivel de seguridad tanto en aplicaciones web como móviles, eliminando la necesidad de almacenar secrets en el cliente [2].

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

3. JWT (JSON Web Token)

Los tokens JWT generados por el Servicio de Autenticación (detallado en la Actividad 2) contienen información sobre la identidad, roles, permisos y tiempo de expiración del usuario, permitiendo que los servicios validen la autorización sin necesidad de mantener sesiones centralizadas [6].

4. API Gateway (Traefik)

Funciona como punto único de entrada para todas las solicitudes dirigidas a los microservicios, siguiendo el patrón documentado por [3]. Centraliza las políticas de seguridad, controla el tráfico y simplifica la administración de la infraestructura. Implementa middleware de autenticación, rate limiting y logging a nivel de infraestructura.

5. Validación de JWT

Este componente verifica la autenticidad, integridad y vigencia de los tokens recibidos. Utiliza la clave pública del servidor de autenticación para verificar la firma RS256 y valida los claims de expiración, issuer y audience. Solo las solicitudes que presentan tokens válidos pueden acceder a los servicios internos.

6. Rate Limiting

Controla la cantidad de solicitudes permitidas por usuario o aplicación durante un periodo de tiempo determinado. Se configuran límites de 100 requests por minuto por usuario con un burst de 50, evitando abusos, proteger la infraestructura y garantizando una distribución equilibrada de los recursos [2].

7. Logging y Monitoreo

Registra todas las solicitudes y eventos relevantes para facilitar la auditoría, el diagnóstico de problemas y la detección de comportamientos sospechosos dentro de la plataforma. Utiliza el stack Prometheus + Grafana + Loki para correlacionar métricas de rendimiento con logs de seguridad.

8. Microservicios de Inventario, Pagos y Logística

Procesan la lógica de negocio principal de la empresa. Cada servicio opera de manera independiente y atiende únicamente las solicitudes previamente validadas por el API Gateway. Implementan autorización a nivel de servicio siguiendo el patrón de PDP/PEP documentado por [3].

Flujo de Seguridad

1. El usuario accede desde el Portal Web o la Aplicación Móvil.
2. El cliente solicita autenticación al Servidor OAuth 2.0 mediante Authorization Code + PKCE.
3. El servidor valida las credenciales, verifica el segundo factor y genera un JWT firmado.
4. El cliente envía solicitudes incluyendo el token JWT en el header Authorization: Bearer.
5. El API Gateway recibe la solicitud, valida el token JWT y verifica la firma.
6. Se aplican las políticas de Rate Limiting según el usuario y el endpoint.
7. La solicitud es registrada mediante los mecanismos de Logging y Monitoreo.
8. El Gateway enruta la petición hacia el microservicio correspondiente.
9. El microservicio realiza autorización a nivel de servicio (RBAC) y procesa la operación.
10. La respuesta es devuelta al cliente con los headers de seguridad correspondientes.

Esta arquitectura mitiga los vectores de ataque críticos: broken authentication (API2), broken access control (API5), unrestricted resource consumption (API4) y security misconfiguration (API8), proporcionando defensa en profundidad para la plataforma de LogiMarket [2].

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

Actividad 5: Sistema de Monitoreo y Auditoría

Diagrama de Flujo del Sistema de Auditoría

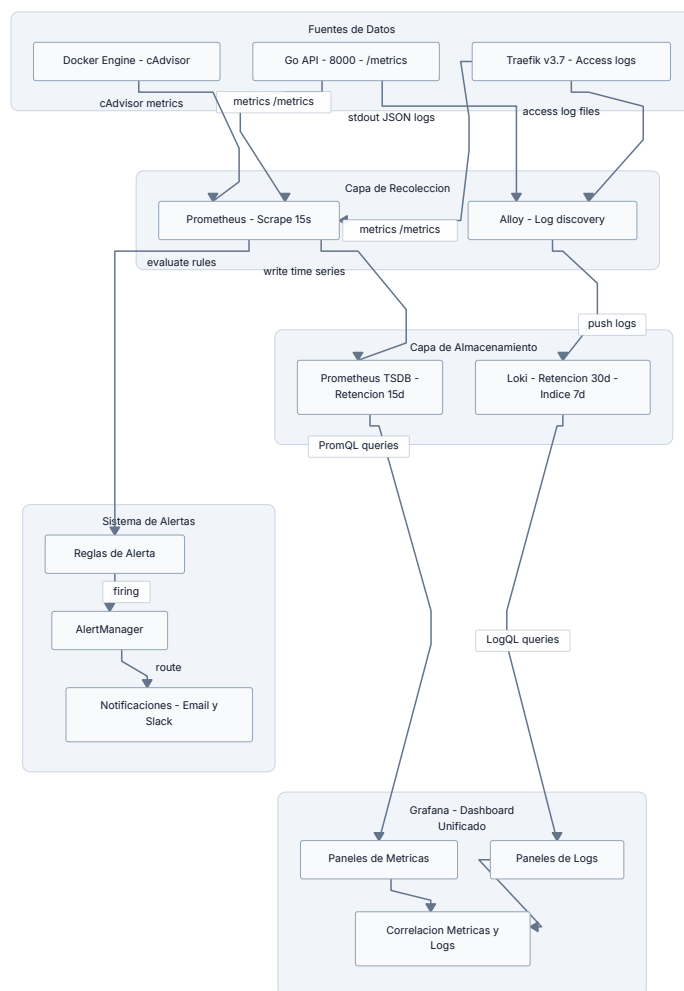


Figura 8: Arquitectura del sistema de observabilidad con recolección, almacenamiento, visualización y alertas.

Descripción Técnica

Contexto

El sistema de auditoría está diseñado para la infraestructura de LogiMarket Perú S.A.C., compuesta por un API Go con Traefik como reverse proxy. El objetivo es implementar un sistema centralizado de monitoreo y registro que permita detectar incidentes de seguridad, cumplir con políticas de retención y proporcionar visibilidad completa sobre el estado del sistema a través de un dashboard unificado en Grafana. Este diseño sigue las recomendaciones del NIST Cybersecurity Framework para las funciones de Detect y Respond [1].

Fuentes de datos

El sistema recopila datos de tres fuentes principales:

- **Go API (puerto 8000):** Expone métricas HTTP en formato Prometheus (/metrics) y genera logs estructurados en JSON incluyendo nivel de log, timestamp, método HTTP, path, status code y latencia de cada request. Los eventos registrados

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 11

incluyen: intentos de acceso, operaciones sobre productos y usuarios, y errores de autenticación. Este enfoque de métricas y logs estructurados es consistente con las prácticas de observabilidad documentadas por [7].

- **Traefik v3.7:** Como API Gateway, genera access logs detallados con información del cliente (IP, User-Agent), ruta solicitada, código de respuesta y tiempo de procesamiento. También expone métricas de requests HTTP, conexiones activas y latencia de backend [8].
- **Docker Engine:** A través de cAdvisor, se recolectan métricas de consumo de recursos de cada contenedor (CPU, memoria, red, disco), permitiendo detectar anomalías como picos de uso que podrían indicar ataques DoS o fugas de memoria.

Capa de recolección

- **Prometheus:** Se configura con un intervalo de scrape de 15 segundos. Contacta periódicamente los endpoints `/metrics` del API Go y de Traefik para obtener métricas en formato de texto. Cada muestra incluye labels como `method`, `path`, `status` y `instance`, lo que permite agregar datos por cualquier dimensión. Prometheus almacena las series temporales en su base de datos TSDB (Time Series Database) con compresión automática [7].
- **Grafana Alloy:** Agente unificado de recolección de datos que reemplaza a Promtail. Descubre automáticamente los contenedores Docker a través del socket, recolecta logs de `stdout/stderr` y los envía a Loki con metadata como el nombre del contenedor, el servicio de compose y el nivel de log. Utiliza el lenguaje de configuración River para definir pipelines de procesamiento que incluyen parsing Docker, extracción de labels y filtrado [9].

Capa de almacenamiento

- **Prometheus TSDB:** Almacena métricas con una retención de 15 días. Este periodo es suficiente para analizar tendencias de corto plazo, detectar picos anómalos y generar alertas en tiempo real. Las métricas se compactan automáticamente y se eliminan las series temporales que exceden la ventana de retención. Prometheus soporta consultas PromQL para calcular tasas (`rate()`), percentiles (`histogram_quantile()`) y agregaciones temporales [7].
- **Loki:** Base de datos de logs indexada por labels (no por contenido completo), con una retención de 30 días para logs y 7 días para el índice invertido. Esta arquitectura permite búsquedas eficientes por labels (servicio, nivel, contenedor) sin el overhead de indexar cada palabra. Los logs se almacenan en bloques comprimidos en object storage. Loki soporta consultas LogQL, un lenguaje similar a PromQL diseñado para filtrar y procesar flujos de logs [9].

Dashboard unificado en Grafana

Grafana se configura como el punto único de visualización, conectando simultáneamente a Prometheus (para métricas) y Loki (para logs). El dashboard incluye:

- **Paneles de métricas:** Tasa de requests por segundo, histograma de latencia (P50, P95, P99), tasa de errores HTTP (4xx, 5xx), uso de recursos por contenedor y uptime del servicio.
- **Paneles de logs:** Stream de logs en tiempo real con filtros por nivel (INFO, WARN, ERROR), búsqueda full-text y visualización de logs estructurados JSON. Permite correlacionar un pico de métricas con los logs específicos que lo causaron.
- **Correlación métricas-logs:** La funcionalidad de “explore” en Grafana permite seleccionar un punto en un gráfico de métricas y ver automáticamente los logs correspondientes a ese rango de tiempo, facilitando la investigación de incidentes [9].

Reglas de alerta

Se definen reglas de alerta basadas en umbrales críticos:

- **Error rate > 5%:** Si más del 5% de los requests retornan código 5xx durante un periodo de 5 minutos, se dispara una alerta indicando un fallo potencial del servicio.
- **Latencia P95 > 2 segundos:** Un aumento sostenido de la latencia del percentil 95 por encima de 2 segundos indica degradación del rendimiento que afecta la experiencia del usuario.
- **Fallos de autenticación > 10/min:** Un volumen inusual de errores de autenticación puede indicar un ataque de fuerza bruta o credenciales comprometidas [2].

Las alertas pasan por AlertManager, que se encarga de agrupar alertas similares, aplicar reglas de enrutamiento (por severidad o servicio) y silenciar notificaciones duplicadas durante ventanas de mantenimiento. Las notificaciones se envían por email, Slack o webhooks configurables.

Políticas de retención

Componente	Tipo de dato	Retención	Justificación
Prometheus	Métricas	15 días	Análisis de tendencias de corto plazo, alertas en tiempo real
Loki	Logs	30 días	Investigación de incidentes, cumplimiento normativo
Loki	Índice invertido	7 días	Búsquedas recientes eficientes, reducción de costo

Figura 2: Políticas de retención de datos del sistema de auditoría

Eventos registrados

Los eventos auditables capturados por el sistema incluyen:

- **Intentos de acceso:** Requests autenticados y no autenticados, con IP de origen y usuario identificado.
- **Operaciones críticas:** Creación, actualización y eliminación de recursos (productos, usuarios, pagos).
- **Errores de autenticación:** Credenciales inválidas, tokens expirados, intentos de acceso con permisos insuficientes [2].
- **Cambios de configuración:** Modificaciones a Traefik, variables de entorno del API y cambios en políticas de seguridad.

Beneficios

- **Cumplimiento normativo:** Los logs retenidos por 30 días permiten responder a auditorías de seguridad y requisitos regulatorios, alineándose con las funciones Identify y Protect del NIST CSF [1].
- **Trazabilidad:** Cada request puede rastrearse desde su origen (IP, usuario) hasta su resultado (status code, latencia), facilitando investigaciones forenses.
- **Detección de incidentes:** Las alertas automáticas permiten respuesta inmediata ante comportamientos anormales, reduciendo el tiempo de detección (MTTD) y el tiempo de respuesta (MTTR).

CUESTIONARIO

Pregunta 1: Una empresa implementa autenticación multifactor, pero continúa sufriendo accesos no autorizados. Analice críticamente qué otros factores organizacionales, tecnológicos y humanos podrían estar influyendo en la ocurrencia de estos incidentes y proponga soluciones integrales.

La implementación de MFA por sí sola no garantiza la eliminación de accesos no autorizados [4], [5]. En el ámbito organizacional, las políticas de acceso condicional mal configuradas (whitelists de IP, geolocalización de confianza) y las cuentas de servicio o contratistas sin MFA habilitado crean vectores de entrada débiles [1]. En el ámbito tecnológico, protocolos legacy como IMAP y POP no soportan MFA, y sistemas en modo “fail-open” permiten acceso sin validación cuando pierden conectividad con el proveedor de identidad [5]. En el ámbito humano, los ataques AiTM interceptan tokens de sesión válidos y el “MFA fatigue” explota la fatiga del usuario enviando solicitudes repetidas [4]. Las soluciones incluyen migración a autenticación passwordless FIDO2/WebAuthn, sistemas “fail-closed” con respaldo local, revisión de políticas de acceso condicional y programas de concienciación [5].

Pregunta 2: La dirección de una empresa considera que implementar mecanismos avanzados de cifrado reducirá significativamente el rendimiento de sus sistemas distribuidos. Evalúe esta afirmación y argumente cómo puede lograrse un equilibrio entre seguridad y desempeño.

La afirmación es una percepción obsoleta [10], [11]. Las extensiones AES-NI en procesadores modernos permiten que AES-GCM opere a velocidades cercanas a la memoria; Google reporta que SSL/TLS representa menos del 1% de carga de CPU y menos de 10 KB de memoria por conexión [10]. TLS 1.3 redujo el handshake de 2 a 1 roundtrip, y con resumiación 0-RTT los clientes recurrentes envían datos en el primer paquete [11]. Twitter confirma que priorizar ECDHE causó aumento insignificante de CPU gracias a keepalive y resumiación de sesión [10]. Para equilibrar seguridad y desempeño se recomiendan certificados ECDSA (256 bits equivale a RSA 3072), OCSP stapling, resumiación con tickets y HSTS [11]. En microservicios, mTLS interno añade solo 1-2ms de latencia, imperceptible frente a WAN de 80-120ms [3].

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 13

Pregunta 3: Si una organización posee recursos limitados para invertir en ciberseguridad, ¿qué controles de seguridad priorizaría para proteger una arquitectura basada en microservicios? Justifique técnicamente su respuesta considerando riesgos, impacto y costo-beneficio.

La priorización debe seguir un enfoque basado en riesgo [1], [3]. Primera prioridad: API Gateway con autenticación centralizada y JWT, que protege todos los microservicios aguas abajo con rate limiting, terminación TLS y logging en una única inversión [3]. Segunda prioridad: Cifrado TLS en tránsito, que con aceleración hardware añade menos del 1% de overhead pero mitiga la interceptación de tráfico [10]. Tercera prioridad: Logging estructurado JSON con Alloy y Prometheus para detección de incidentes con inversión mínima [9]. Cuarta prioridad: Gestión de secretos (hash Argon2/bcrypt, rotación de tokens) que mitiga exposición de credenciales [2]. Quinta prioridad: Segmentación de red Docker (volúmenes read-only, usuarios no-root) sin costo adicional [3]. Los controles a diferir incluyen SIEM comerciales (usar ELK), WAF dedicado (Traefik incluye protecciones básicas) y pentest automatizado (OWASP ZAP trimestral).

CONCLUSIONES

CONCLUSIONES

1. La implementación de un sistema de seguridad integral para LogiMarket Perú S.A.C. demostró que la combinación de cifrado TLS con certificados autofirmados, reverse proxy con Traefik y un stack de monitoreo centralizado proporciona una defensa en profundidad efectiva contra los vectores de ataque identificados en el OWASP API Security Top 10 [2].
2. El diseño de la arquitectura de autenticación con MFA, gestión de identidades y tokens JWT, siguiendo las directrices del NIST Cybersecurity Framework [1], establece un modelo de confianza cero que mitiga los riesgos de accesos no autorizados y credenciales comprometidas descritos por [4].
3. El stack de monitoreo y auditoría compuesto por Prometheus, Grafana, Loki y AlertManager permite la detección en tiempo real de incidentes de seguridad, cumpliendo con los principios de registro y auditoría establecidos por [2] y proporcionando trazabilidad completa de cada request a través del sistema.
4. La evaluación del impacto del cifrado en el rendimiento confirma que TLS 1.3 con aceleración por hardware (AES-NI) añade menos del 1% de overhead en CPUs modernas, refutando la creencia de que el cifrado degrada significativamente el rendimiento [10], [11].

RECOMENDACIONES

1. Implementar un Identity Provider centralizado como Keycloak o Auth0 para consolidar la autenticación y autorización, siguiendo el patrón de autenticación a nivel de borde (edge-level) recomendado por [3], eliminando la duplicación de lógica de autenticación en cada microservicio.
2. Migrar los certificados autofirmados a una PKI interna con CA propia para producción, y considerar la integración de Let's Encrypt para ambientes de staging, siguiendo las mejores prácticas de gestión de certificados documentadas por [12].
3. Establecer políticas de retención de datos alineadas con requisitos regulatorios específicos del sector fintech en Perú, y configurar reglas de alerta en Prometheus y Loki adaptadas a los patrones de tráfico de LogiMarket para reducir la tasa de falsos positivos.
4. Implementar rate limiting y circuit breakers a nivel de API Gateway (Traefik) para proteger los microservicios frente a ataques DoS/DDoS, complementando las métricas de monitoreo existentes con métricas de negocio específicas.

REFERENCIAS

Bibliography

- [1] National Institute of Standards and Technology, "NIST Cybersecurity Framework 2.0." [Online]. Available: <https://www.nist.gov/cyberframework>
- [2] OWASP Foundation, "OWASP API Security Top 10 – 2023." [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0x00-header/>

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 14

- [3] OWASP Foundation, "Microservices Security Cheat Sheet." [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Microservices_Security_Cheat_Sheet.html
- [4] Bugcrowd, "MFA Security Part 1: How Attackers Bypass Multi-Factor Authentication." [Online]. Available: <https://www.bugcrowd.com/blog/mfa-security-part-1-how-attackers-bypass-multi-factor-authentication/>
- [5] RSA Security, "How Attackers Bypass MFA and What Security Teams Can Learn." [Online]. Available: <https://www.rsa.com/resources/blog/multi-factor-authentication/how-attackers-bypass-mfa-and-how-to-reduce-risk-rsa/>
- [6] M. Jones, J. Bradley, and N. Sakimura, "RFC 7519 – JSON Web Token (JWT)." [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7519>
- [7] Prometheus Authors, "Prometheus – Monitoring System & Time Series Database." [Online]. Available: <https://prometheus.io/docs/>
- [8] Traefik Labs, "Traefik Proxy Documentation." [Online]. Available: <https://doc.traefik.io/traefik/>
- [9] Grafana Labs, "Grafana Loki Documentation – Log Aggregation System." [Online]. Available: <https://grafana.com/docs/loki/latest/>
- [10] Ilya Grigorik, "Is TLS Fast Yet?." [Online]. Available: <https://istlsfastyet.com/>
- [11] EaseCloud, "How SSL/TLS Impacts Performance and How to Optimize It." [Online]. Available: <https://blog.easecloud.io/cloud-infrastructure/optimization-and-performance-impact-of-ssl-tls/>
- [12] OpenSSL Software Foundation, "OpenSSL Documentation." [Online]. Available: <https://docs.openssl.org/master/>